

Module 3

UART — Protocol + RTL + Basic Testbench

Learning objectives

- UART 8N1: Start bit (0), 8 data bits (LSB first), stop bit (1); idle high.
- Baud rate: Role of a baud generator (clock divider → baud_tick).
- RTL architecture: TX (parallel→serial, shift per baud_tick), RX (start detect, sample, data_valid),...
- Basic TB: Loopback, directed stimulus, and checking without UVM.

Prerequisites

- Module 1 (spec → RTL) and Module 2 (basic TB → UVM, toolchain).
- Verilator, Make, C++ compiler. No UVM required for the uart_baseline example.

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["UART Protocol (8N1)"]

T2["RTL Architecture"]

T1 --> T2

T3["Basic Testbench"]

T2 --> T3

Understand the UART protocol (8N1, baud), translate it to RTL (TX, RX, baud gen), and verify with a **basic** (non-UVM) directed testbench.

Overview

- Module 3 is the first **protocol** module:

Syllabus topics

1. UART Protocol (8N1) (1/4)

- Signals: Serial line (tx/rx), idle high; system clock; baud_tick (one pulse per bit time).
- Frame: 1 start bit (0), 8 data bits (LSB first), 1 stop bit (1).
- Timing: One bit per baud_tick; baud_tick derived from system clock via a divider (e.g. $DIVIDER = clk_hz / baud$).
- Framing: `idle=1` → `start=0` → 8 data bits (LSB first) → `stop=1`.
- Baud math: $DIVIDER = round(clk_hz / baud)$.
Example: 50 MHz clock, 115200 baud → divider ≈ 434 .
- Reset state: Line idles high, TX not busy, RX clears internal shift register, `data\_valid` low.

1. UART Protocol (8N1) (2/4)

- Errors: ``framing_error`` when stop bit is not high at expected sample; optional ``parity_error`` if pa...
- Inputs: ``clk``, ``rst_n``, ``baud_tick``, ``start``,
``data_in[7:0]``.
- Outputs: ``tx``, ``busy``.
- Behavior: On ``start``, latch ``data_in``, drive start bit low, then shift out bits [0:7] each ``baud_ti...`
- Edge cases: Ignore ``start`` while ``busy`` (or queue it in a FIFO if you extend the design).
- Inputs: ``clk``, ``rst_n``, ``baud_tick``, ``rx``.

1. UART Protocol (8N1) (3/4)

- Outputs: ``data_out[7:0]``, ``data_valid`` (pulse), ``framing_error``.
- Robustness options: Add metastability sync on ``rx``; add oversampling + majority vote; add ``rx_ready``...
- Inputs: ``clk``, ``rst_n``, ``divisor`` (constant parameter in the baseline).
- Output: ``baud_tick`` high for one cycle every ``divisor`` clocks.
- For synthesis, keep the counter simple (``divisor-1`` down-counter or up-counter compare). For sim, y...
- Data bits: 5–9 data bits are common.

1. UART Protocol (8N1) (4/4)

- Parity: Even/odd/mark/space; adds a parity bit before stop.
- Stop bits: 1 or 2 stop bits.
- Flow control: RTS/CTS hardware pins (outside the serial line) to throttle traffic.
- The baseline RTL assumes 8N1 with no parity and no flow control.

2. RTL Architecture

- baud_gen: Divides `clk`; outputs `baud\_tick` one cycle every DIVIDER cycles.
- uart_tx: On `start`, loads `data\_in` into shift register; outputs one bit per `baud\_tick` (start, d...
- uart_rx: Detects start bit (line low), samples one bit per `baud\_tick`, reconstructs byte; pulses `...

3. Basic Testbench

- Loopback: Connect TX output to RX input; send bytes from TX, check they appear on RX.
- Directed test: Reset release, send 0x55 and 0xAA, wait for `data\_valid`, check `data\_out`. Implemen...

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --check
```

Terminal: module3_check

```
[[0;36m===== [[0m
[[0;36mModule 3: Demo commands (UART baseline) [[0m
[[0;36m===== [[0m
```

From repo root:

```
# 1) Environment sanity:
verilator --version
make --version
```

```
# 2) Run the UART baseline example (loopback, basic TB, no UVM):
cd module3/examples/uart_baseline
make run
```

```
[[1;33m[INFO] [[0m See module3/EXAMPLES.md and module3/examples/uart_baseline/README.md for more.
```

Example 1: UART baseline

- ``dut/`: `uart_tx.v`, `uart_rx.v`, `baud_gen.v``
- ``top_uart_baseline.sv``: Top with loopback; directed stimulus and checks in initial blocks.
- ``sim_main.cpp``: C++ harness (clock, reset); simulation runs until ``$finish``.

Demo: UART baseline

```
$ ./scripts/module3.sh --run
```

Terminal: ex01_uart_baseline

```
make run
```

```
Expected: `[PASS] UART baseline: 0x55 loopback OK`, `[PASS] UART baseline: 0xAA loopback OK`, then `
```

Practice & assessment

Exercises

- Run `uart_baseline`
- Trace one byte
- Baud and divider
- Optional: Waveforms

Assessment checklist

- Can describe UART 8N1 (start bit, 8 data LSB first, stop bit) and the role of baud_tick.
- Can explain the role of uart_tx, uart_rx, and baud_gen in the RTL.
- Can run uart_baseline and interpret the loopback test result.
- Ready for Module 4: UART UVM+SV verification.

Summary & next steps

- Understand the UART protocol (8N1, baud), translate it to RTL (TX, RX, baud gen), and verify with a...
- Complete module3/CHECKLIST.md
- Run `./scripts/module3.sh --check`
- Next: UART UVM+SV